EasyMyLearning

Task Manager - Architecture & Deployment Plan

Next.js + React frontend | Go backend | PostgreSQL | Docker | REST API

Purpose	Convert the professional task-management UI prototype into a configurable production application with a clean frontend/backend separation and an extensible database model.

Recommended product shape

- Responsive task dashboard for desktop, tablet, and mobile.
- Configurable statuses, priorities, task categories, dashboard widgets, branding, and custom fields.
- Go API as the business and authorization layer; Next.js as the application/UI layer.
- PostgreSQL as the source of truth, with optional Redis and object storage for scale.
- Dockerized local/staging/production deployment with CI/CD.

Key implementation principle: anything an administrator may reasonably want to change later should be represented as configuration/data instead of being hardcoded into the frontend.

1. Recommended system architecture

The application should use a layered architecture so the frontend stays focused on presentation while all business rules, validation, and data access remain in the Go backend.

```
Users
  |
  v
Next.js / React
  |
  | HTTPS / JSON
  v
Go REST API
  |
  +--> PostgreSQL
  |
  +--> Redis (optional: cache / jobs)
  |
  +--> S3 / Cloudflare R2 (attachments)
```

Layer	Recommended technology	Primary responsibility
Frontend	Next.js + React + TypeScript	Dashboard, tasks, forms, filters, responsive UI, admin screens
Backend	Go	Authentication, authorization, validation, business logic, API, audit
Database	PostgreSQL	Tasks, users, configuration, workflow, audit, notifications
Cache / jobs	Redis	Caching, queues, scheduled jobs when needed
Object storage	S3 / Cloudflare R2	Task attachments and uploaded files
Proxy / edge	Nginx + Cloudflare	TLS termination, routing, CDN, basic protection

Recommended domains

```
app.easymylearning.com  -> Next.js application
api.easymylearning.com  -> Nginx -> Go API
files.easymylearning.com -> object storage / signed file URLs
```

2. Frontend - Next.js / React

The prototype should be converted into reusable components rather than kept as one large HTML file.

```
frontend/
  app/
    dashboard/
    tasks/
    categories/
    settings/
    admin/
  components/
    layout/
    dashboard/
    tasks/
    ui/
  lib/
    api.ts
  types/
    task.ts
```

Recommended frontend stack

Area	Choice
Framework	Next.js + React + TypeScript
UI	Tailwind CSS or a lightweight component system
Icons	Lucide React / standard SVG icon library
Forms	React Hook Form
Validation	Zod
Server state	TanStack Query
API contract	OpenAPI-generated or typed client layer

Current UI capabilities to preserve

- Not Started -> In Progress -> Completed workflow.
- Inline priority and status editing.
- Task Category, due dates, overdue filtering, search and sorting.
- Edit task details and delete actions.
- Responsive desktop, tablet, and mobile behavior.
- EasyMyLearning primary brand color #FFAA00 with semantic status colors.

3. Backend - Go REST API

Keep handlers thin, place business rules in services, and use repositories/query functions for database interaction.

```
backend/
  cmd/api/main.go
  internal/
    config/
    db/
    httpapi/
    task/
    auth/
    user/
    dashboard/
    notification/
    configuration/
  middleware/
```

Core endpoints

Group	Endpoint examples
Tasks	GET /api/v1/tasks | POST /api/v1/tasks | GET /api/v1/tasks/{id}
Task update	PUT /api/v1/tasks/{id} | DELETE /api/v1/tasks/{id}
Inline updates	PATCH /api/v1/tasks/{id}/status | /priority | /category | /due-date
Dashboard	GET /api/v1/dashboard/summary | /dashboard/upcoming
Configuration	GET /task-categories | /task-statuses | /task-priorities
Public config	GET /api/v1/config/public
Widgets	GET /api/v1/dashboard/widgets

Example task payload

```
{
  "title": "Prepare monthly report",
  "description": "Finalize KPI report",
  "category_id": "uuid",
  "status_id": "uuid",
  "priority_id": "uuid",
  "assigned_to": "uuid",
  "due_date": "2026-10-05"
}
```

4. PostgreSQL database model

Use normalized tables for workflow and configuration. This prevents frontend assumptions from becoming permanent database rules.

Table	Purpose
users	User accounts and profile information
roles / user_roles	Role-based access control foundation
task_categories	Configurable Task Category values
task_statuses	Not Started, In Progress, Completed, plus future states
task_priorities	Low, Medium, High, Urgent and future priority levels
tasks	Main task record and relationships
task_checklists	Sub-items within a task
task_comments	Collaboration / discussion
task_attachments	File metadata for object storage
task_activity_logs	Audit trail of task changes
notifications	In-app notifications
user_preferences	Theme, timezone, view, pagination and notification preferences
dashboard_widgets	Configurable dashboard widgets
user_dashboard_widgets	User-specific widget layout and visibility
custom_fields	Admin-defined dynamic fields
custom_field_values	Values of dynamic fields
system_settings	Branding and application configuration

Main relationships

```
tasks
  |-- status_id   -> task_statuses.id
  |-- priority_id -> task_priorities.id
  |-- category_id -> task_categories.id
  |-- created_by  -> users.id
  `-- assigned_to -> users.id
```

5. Make the UI configurable

The strongest architectural improvement is to move changeable UI behavior into database-backed configuration.

Feature	Database-driven?	Example
Primary branding	Yes	#FFAA00
Task Category	Yes	Work, Study, Admissions, Finance
Status	Yes	Not Started, In Progress, Blocked, Completed
Priority	Yes	Low, Medium, High, Urgent
Dashboard cards	Yes	Total, Active, Completed, Overdue
Dashboard layout	Yes	Per-user widget position / size
Custom task fields	Yes	Follow-up Date, Reference Number
User preferences	Yes	Theme, timezone, default task view

Admin configuration screens

- Users and Roles & Permissions.
- Task Categories, Statuses and Priorities.
- Dashboard Widgets and layouts.
- Custom Fields.
- Branding and System Settings.
- Audit Logs and operational settings.

This lets an administrator add a new task category or workflow state without requiring a frontend rebuild.

6. Additional features worth adding

Feature	Why it matters
Task Details Drawer	Keeps full description, checklist, comments, attachments and activity accessible without leaving the task list.
Kanban View	Provides a visual workflow for Not Started / In Progress / Completed.
Bulk Actions	Change status, priority, category, assignee or archive multiple tasks together.
Saved Views	Save reusable combinations such as Overdue Work or High Priority.
Task Assignment	Supports team ownership and accountability.
Checklist	Break larger tasks into measurable steps.
Comments	Adds team collaboration context.
Activity / Audit Log	Shows who changed status, priority or other task data.
Notifications	Alerts for assignment, due dates, overdue items and comments.
Keyboard shortcuts	Faster desktop workflow for power users.

Recommended task detail experience

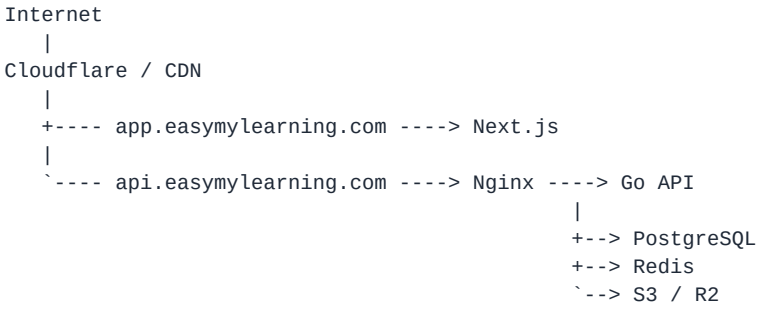
```
Task Details
----------------------------------------
Title
Description
Status       In Progress
Priority     High
Task Category Operations
Due Date     05 Oct

Checklist
[x] Collect data
[x] Verify figures
[ ] Finalize charts

Activity
- Priority changed
- Status changed

Comments
- Team notes and discussion
```

7. Deployment architecture



Containerized services

Service	Container / target
Frontend	Next.js container or Vercel deployment
Backend	Go container behind Nginx
Database	Managed PostgreSQL preferred for production
Redis	Managed Redis or private container
Reverse proxy	Nginx
Edge / DNS	Cloudflare
Files	S3 / Cloudflare R2

Docker local startup

```
cp .env.example .env
docker compose up --build

Frontend: http://localhost:3000
API:      http://localhost:8080
Health:   http://localhost:8080/health
```

Production split

- Staging: staging.easymylearning.com and api-staging.easymylearning.com.
- Production: app.easymylearning.com and api.easymylearning.com.
- Run database backups and use a managed PostgreSQL service where possible.
- Keep secrets outside source control and inject them through deployment secrets.

8. CI/CD, security and operations

A production deployment should introduce operational controls before public release.

CI/CD

- GitHub repository with develop/staging and main/production branches.
- Run frontend build, TypeScript checks, Go tests and linting on pull requests.
- Build Docker images and deploy automatically after approval.
- Run migrations as a controlled deployment step.

Security baseline

- HTTPS everywhere.
- Secure session or JWT authentication with refresh strategy.
- Role-based authorization enforced in Go, not only hidden in the frontend.
- Request validation, rate limiting, CORS policy and secure HTTP headers.
- Password hashing and secret management.
- Soft delete for business records where auditability matters.
- Regular PostgreSQL backups and restore testing.

Production note

The starter package is intentionally focused on the core task workflow and extensible schema. Authentication/RBAC, production file storage, advanced configuration CRUD, observability and infrastructure hardening should be completed before public production use.

9. Recommended implementation phases

Phase	Scope	Outcome
Phase 1	Foundation: auth, users, roles, categories, statuses, priorities, tasks	Stable core domain and permissions model
Phase 2	Convert current UI to Next.js; wire dashboard/task APIs	Production-ready responsive task workspace
Phase 3	Details drawer, checklist, comments, attachments, activity, assignees	Complete collaborative task product
Phase 4	Dashboard widgets, custom fields, system settings, saved views, audit log	Configurable EasyMyLearning platform
Phase 5	Docker, Nginx, CI/CD, backups, monitoring, security hardening	Deployable production system

Suggested first milestone

The first production milestone should deliver the dashboard, task CRUD, inline status/priority/category updates, search, sorting, overdue handling, authentication, RBAC, and the configurable reference tables (categories, statuses, priorities).

10. Full-stack starter repository

```
easymylearning-task-manager/
|
|-- frontend/                Next.js / React
|-- backend/                 Go REST API
|-- database/
|     |-- migrations/        PostgreSQL schema
|     `-- seeds/             Demo/config seed data
|-- deploy/nginx/            Nginx configuration
|-- scripts/
|     |-- migrate.sh
|     |-- seed.sh
|     `-- backup.sh
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

Database initialization

```
psql "$DATABASE_URL" -f database/migrations/001_init.sql
psql "$DATABASE_URL" -f database/seeds/001_seed.sql
```

Backup

```
pg_dump "$DATABASE_URL" > backups/easymylearning_tasks_TIMESTAMP.sql
```

API documentation

Keep the OpenAPI file versioned alongside the Go API. It becomes the contract between the Next.js frontend and backend and can later generate typed clients, API docs and integration-test fixtures.